

ZAGAZIG UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER AND SYSTEMS ENGINEERING DEPARTMENT
CSE 226: Hardware Design

Cipher-16

A 16-bit Cryptographic Coprocessor in VHDL

Project Report

Submitted: May 16, 2026

Team Members

Mohamed Emad ElSawy
Mohamed Mahmoud Ramadan
Mohamed Ali Mohamed
Mohamed Magdy Hendawy
Mohamed Mustafa ElSayed

Course CSE 226: Hardware Design
Tools GHDL, GTKWave, Active-HDL
HDL VHDL (IEEE 1076)

Contents

1	The Assignment Aim	2
2	The Problem Solution	2
2.1	Inputs and Outputs	3
3	Implementation	4
3.1	Source Modules	4
3.2	Schematic Diagram (Active-HDL)	5
3.3	Simulation Results	11

1 The Assignment Aim

The assignment is to design and verify a 16-bit cryptographic coprocessor in VHDL. The coprocessor is the datapath of a small hardware accelerator that performs the inner operations used by block ciphers: register-to-register reads, non-linear substitution through S-Boxes, arithmetic and logical operations through an ALU, and bit-level rotations through a shifter.

The work covers the full RTL flow:

- Partition the design into reusable components (register file, non-linear lookup unit, shifter, ALU, combinational block, top-level coprocessor).
- Define a single 4-bit CTRL encoding that selects between the functional units.
- Wire the components together inside a clocked top-level entity with an asynchronous reset and an input register that stages the operand and opcode fields.
- Verify each block with its own testbench, then verify the integrated coprocessor with a self-checking testbench that exercises reset, LUT substitution, write-back, the ALU path, the shifter path, and reset-while-running behaviour.

The deliverables are the synthesisable VHDL source for each block, a schematic of the top-level entity captured in Active-HDL, simulation waveforms produced by GHDL/GTKWave, and this report.

2 The Problem Solution

The coprocessor is organised as two cooperating blocks:

1. A **datapath** consisting of a 16×16 register file preceded by an input pipeline register. The input register captures Ra, Rb, Rd and CTRL on the rising clock edge; the register file exposes the two operand registers on A_BUS and B_BUS, and writes the previous-cycle result back to R[Rd].
2. A **combinational logic block** that drives the result bus from one of three functional units (a non-linear lookup operation unit with two S-Boxes, an ALU, and a shifter), under control of a small decoder that interprets CTRL.

The functional unit driving RESULT is chosen by the decoder according to Table 1.

Table 1: CTRL routing in the combinational block.

CTRL[3]	CTRL[1:0]	Selected unit
0	–	Non-Linear Lookup (S-Boxes)
1	11	Shifter
1	other	ALU

The shifter recognises three opcodes; any other code drives a zero output (Table 2).

Table 2: Shifter opcodes.

CTRL	Operation	Effect on B_BUS
1000	ROR8	8-bit right rotation
1001	ROR4	4-bit right rotation
1010	SLL8	8-bit left shift, zero-filled
other	(none)	SHIFT_OUT = 0x0000

The non-linear lookup unit splits the low 8 bits of **A_BUS** into two 4-bit halves: the lower half is mapped through S-Box 1, the upper half through S-Box 2 (Table 3). The two 4-bit outputs are concatenated and merged with the unchanged high byte of **A_BUS** to produce the 16-bit result.

Table 3: S-Box substitution tables (4-bit $\rightarrow$ 4-bit).

S-Box 1		S-Box 2	
Input	Output	Input	Output
0000	0001	0000	1111
0001	1011	0001	0000
0010	1001	0010	1101
0011	1100	0011	0111
0100	1101	0100	1011
0101	0110	0101	1110
0110	1111	0110	0101
0111	0011	0111	1010
1000	1110	1000	1001
1001	1000	1001	0010
1010	0111	1010	1100
1011	0100	1011	0001
1100	1010	1100	0011
1101	0010	1101	0100
1110	0101	1110	1000
1111	0000	1111	0110

2.1 Inputs and Outputs

The top-level entity **coprocessor** exposes a register-mapped control interface; all data resides in the internal register file and is read or written through register indices.

Table 4: Coprocessor port list.

Port	Dir	Width	Function
clock	in	1	System clock. All sequential elements update on the rising edge.
reset	in	1	Asynchronous, active-high. Clears the input register and zeroes every register-file location.
CTRL	in	4	Opcode. Selects the functional unit and (for ALU/Shifter) the operation.
Ra	in	4	Index of the source-A register. Drives A_BUS .
Rb	in	4	Index of the source-B register. Drives B_BUS .
Rd	in	4	Index of the destination register. Receives the previous-cycle RESULT .
RESULT	out	16	Combinational output of the selected functional unit for the current operands.

Internal buses. **A_BUS** and **B_BUS** are 16-bit signals between the register file and the combinational block; they are not exposed at the top-level ports. The non-linear lookup unit only consumes **A_BUS**[7:0]; the upper byte is forwarded unchanged.

Pipeline behaviour. Because the operand fields pass through the input register, an instruction issued on edge N produces its **RESULT** during cycle N and is written back to **R[Rd]** on edge $N+1$. A read of a register that was the destination of the previous instruction therefore returns the value written one cycle earlier.

3 Implementation

The design is split across five VHDL files in `src/`, each defining one entity. The top-level coprocessor entity instantiates the register file and the combinational block; the combinational block in turn instantiates the lookup unit, the ALU and the shifter.

3.1 Source Modules

Table 5: Source files and the entities they define.

File	Role
<code>coprocessor.vhd</code>	Top entity. Input register + register file + combinational block.
<code>register_file.vhd</code>	16×16 register array. Combinational reads, synchronous write.
<code>combinational_circuit.vhd</code>	Functional-unit MUX driven by the CTRL decoder.
<code>non_linear_lookup.vhd</code>	Two-S-Box substitution on the low byte of A_BUS.
<code>shifter.vhd</code>	Rotation/shift unit driven by CTRL.
<code>alu.vhd</code>	Arithmetic Logic Unit.

The top entity stages every input through a clocked register before driving the rest of the datapath. This guarantees that the operand indices and the opcode are stable for a full cycle while the combinational block settles.

```

1  entity coprocessor is
2      port (
3          clock    : in  std_logic;
4          reset    : in  std_logic;
5          CTRL     : in  std_logic_vector(3 downto 0);
6          Ra       : in  std_logic_vector(3 downto 0);
7          Rb       : in  std_logic_vector(3 downto 0);
8          Rd       : in  std_logic_vector(3 downto 0);
9          RESULT   : out std_logic_vector(15 downto 0)
10     );
11 end coprocessor;
12
13 architecture rtl of coprocessor is
14     signal Ra_reg, Rb_reg, Rd_reg, CTRL_reg : std_logic_vector(3 downto 0);
15     signal A_BUS, B_BUS, result_s          : std_logic_vector(15 downto 0);
16 begin
17     input_reg: process(clock, reset)
18     begin
19         if reset = '1' then
20             Ra_reg <= (others => '0'); Rb_reg <= (others => '0');
21             Rd_reg <= (others => '0'); CTRL_reg <= (others => '0');
22         elsif rising_edge(clock) then
23             Ra_reg <= Ra; Rb_reg <= Rb; Rd_reg <= Rd; CTRL_reg <= CTRL;
24         end if;
25     end process;
26
27     reg_file: entity work.register_file port map (
28         clk=>clock, rst=>reset, RdWEn=>'1',
29         Ra=>Ra_reg, Rb=>Rb_reg, Rd=>Rd_reg,
30         Res=>result_s, SRCa=>A_BUS, SRCb=>B_BUS);
31
32     comb_logic: entity work.combinational_circuit port map (
33         ABUS=>A_BUS, BBUS=>B_BUS, CTRL=>CTRL_reg, Result=>result_s);
34
35     RESULT <= result_s;
36 end architecture;

```

Listing 1: Top-level entity (src/coprocessor.vhd, abridged).

The decoder inside `combinational_circuit` matches the routing rules in Table 1.

```
1 control_logic: process(CTRL, tmp_out1, tmp_out2, tmp_out3)
2 begin
3     case CTRL(3) is
4         when '0' => RESULT <= tmp_out1;           -- LUT path
5         when others =>
6             case CTRL(1 downto 0) is
7                 when "11" => RESULT <= tmp_out3;   -- Shifter path
8                 when others => RESULT <= tmp_out2; -- ALU path
9             end case;
10        end case;
11    end process;
```

Listing 2: CTRL decoder driving the result MUX.

3.2 Schematic Diagram (Active-HDL)

The schematics in this subsection are the RTL views exported after elaborating each entity. Figure 1 is the top-level view that ties the four input registers, the register file, and the combinational block together; the remaining figures zoom into each sub-block.

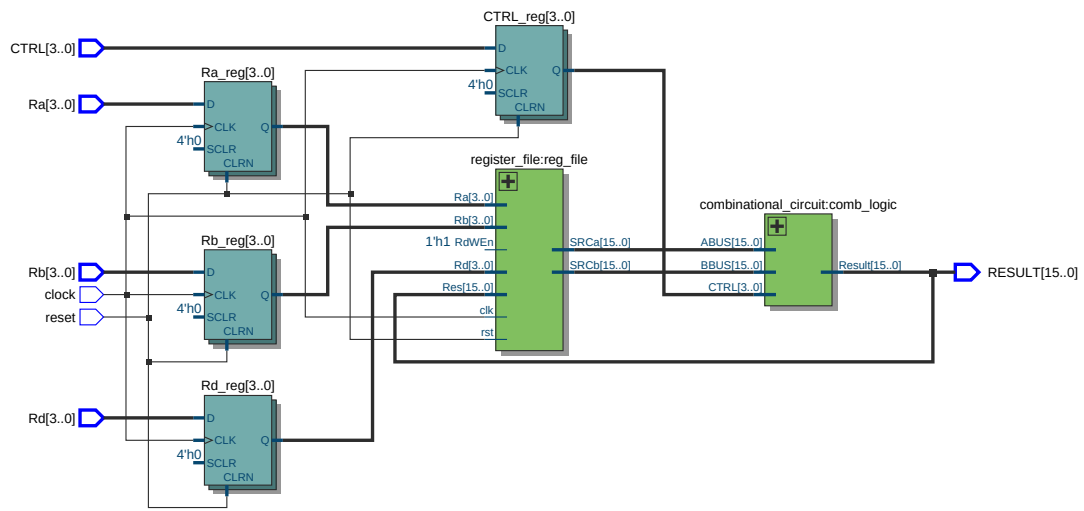


Figure 1: Top-level coprocessor schematic: input pipeline registers, register file, combinational block.

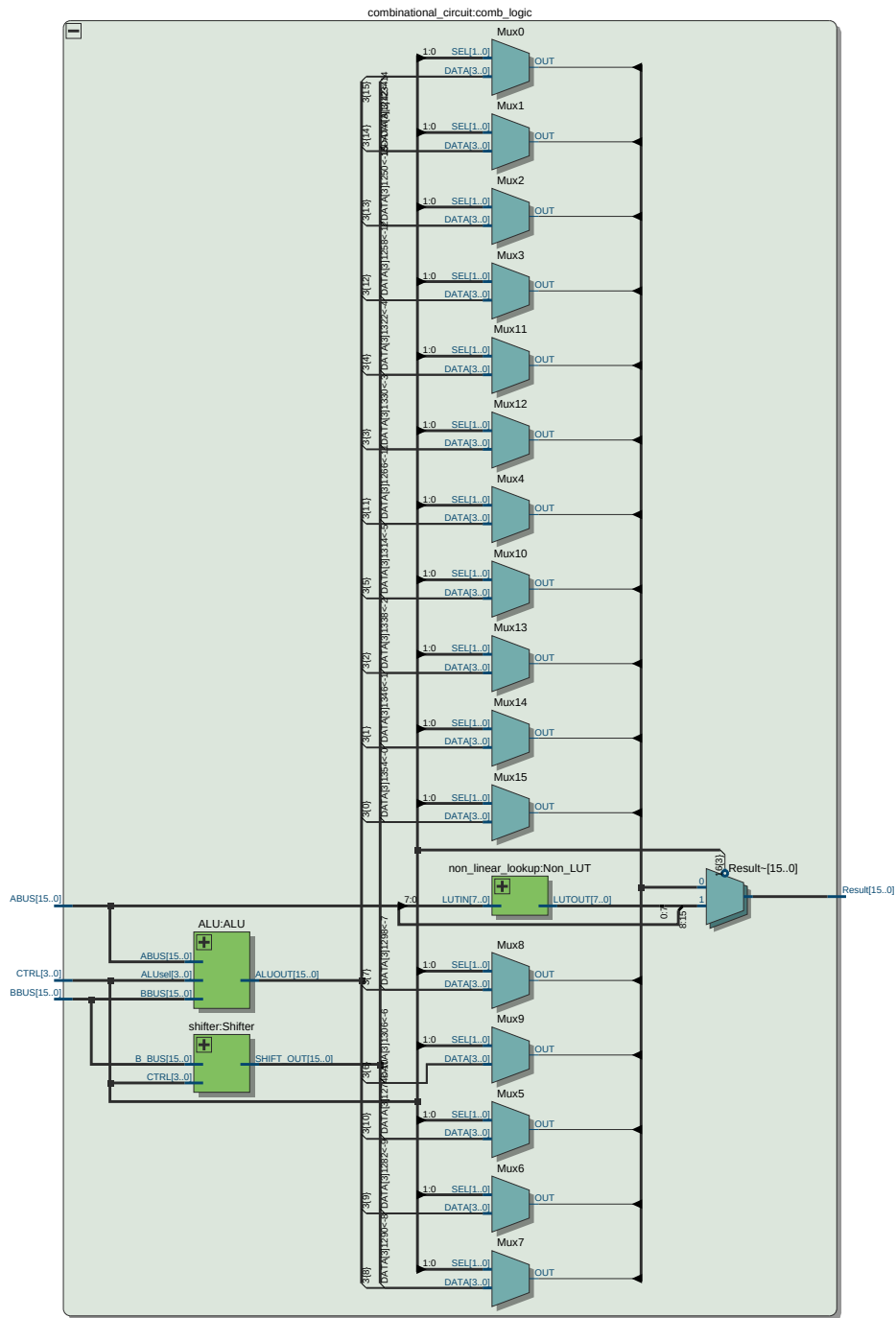


Figure 2: Combinational logic block: ALU, LUT, Shifter, and the per-bit result MUX driven by the CTRL decoder.

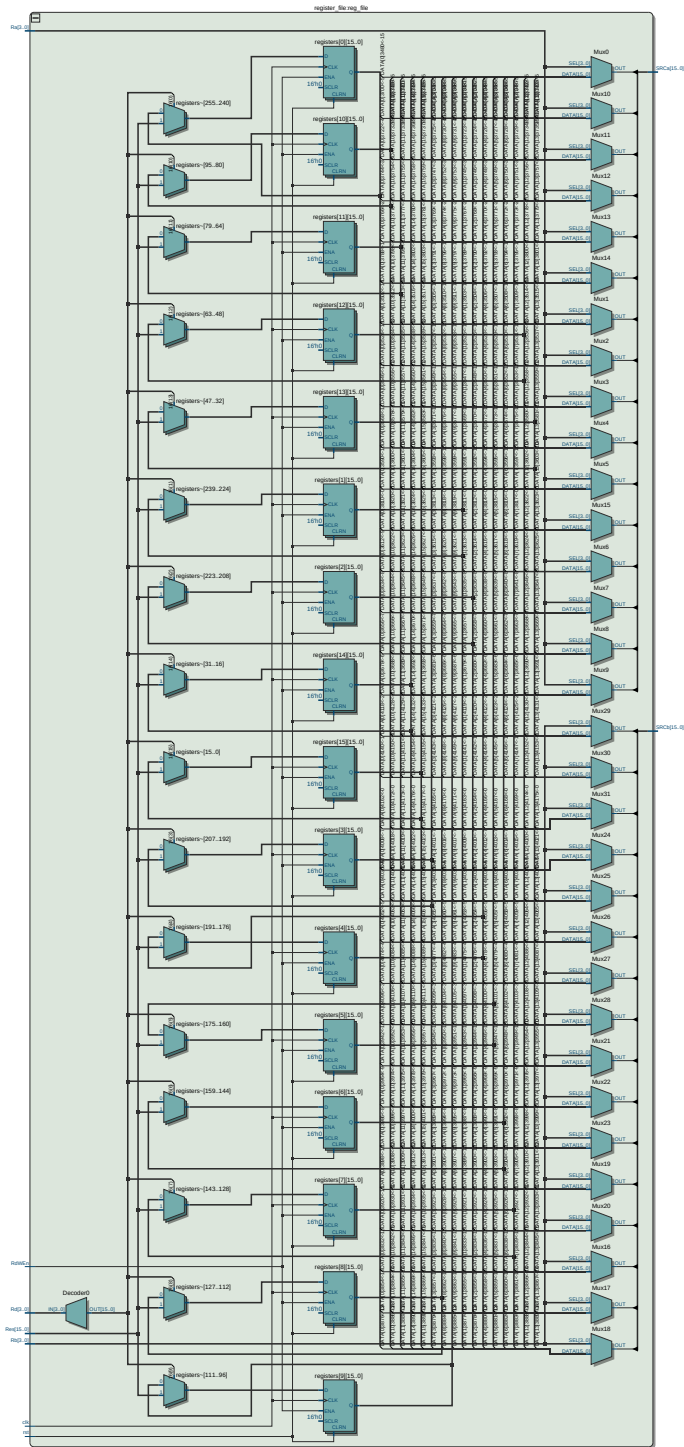


Figure 3: 16×16 register-file RTL view: synchronous write port and two combinational read ports.

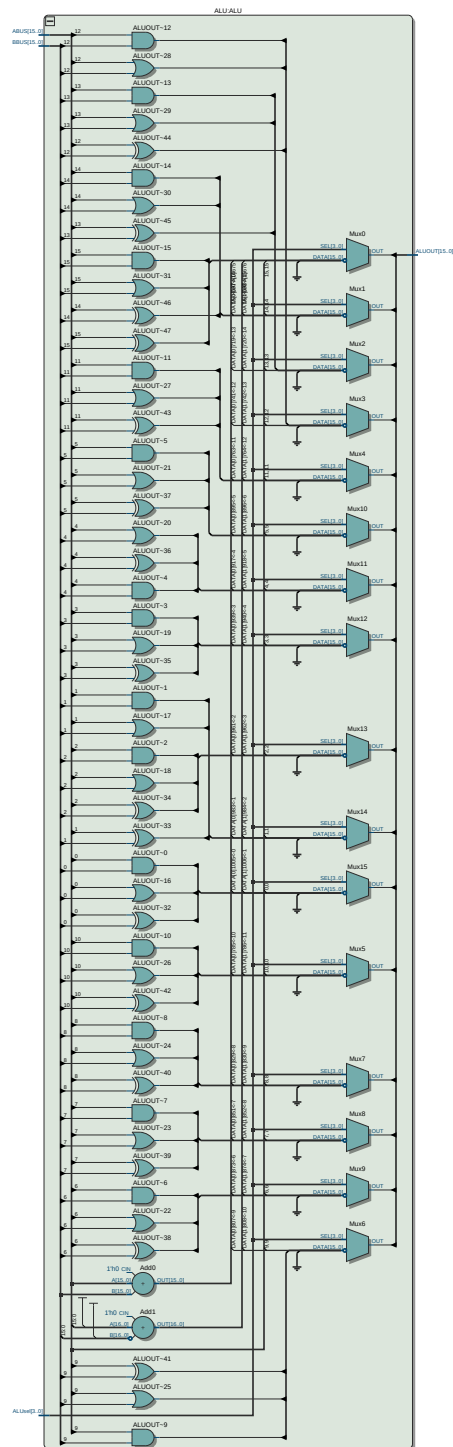


Figure 4: ALU RTL view.

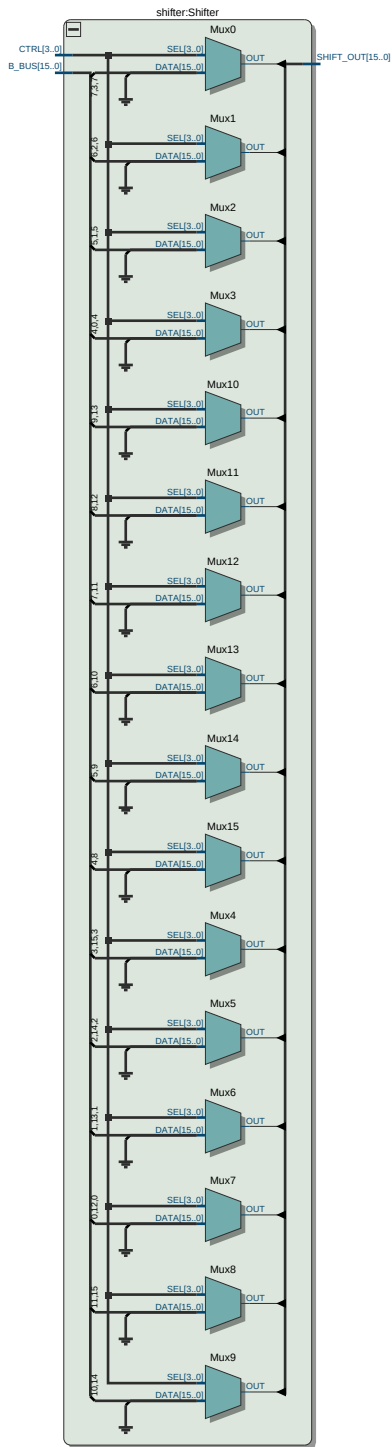


Figure 5: Shifter RTL view: bit-wise routing for ROR8, ROR4, and SLL8.

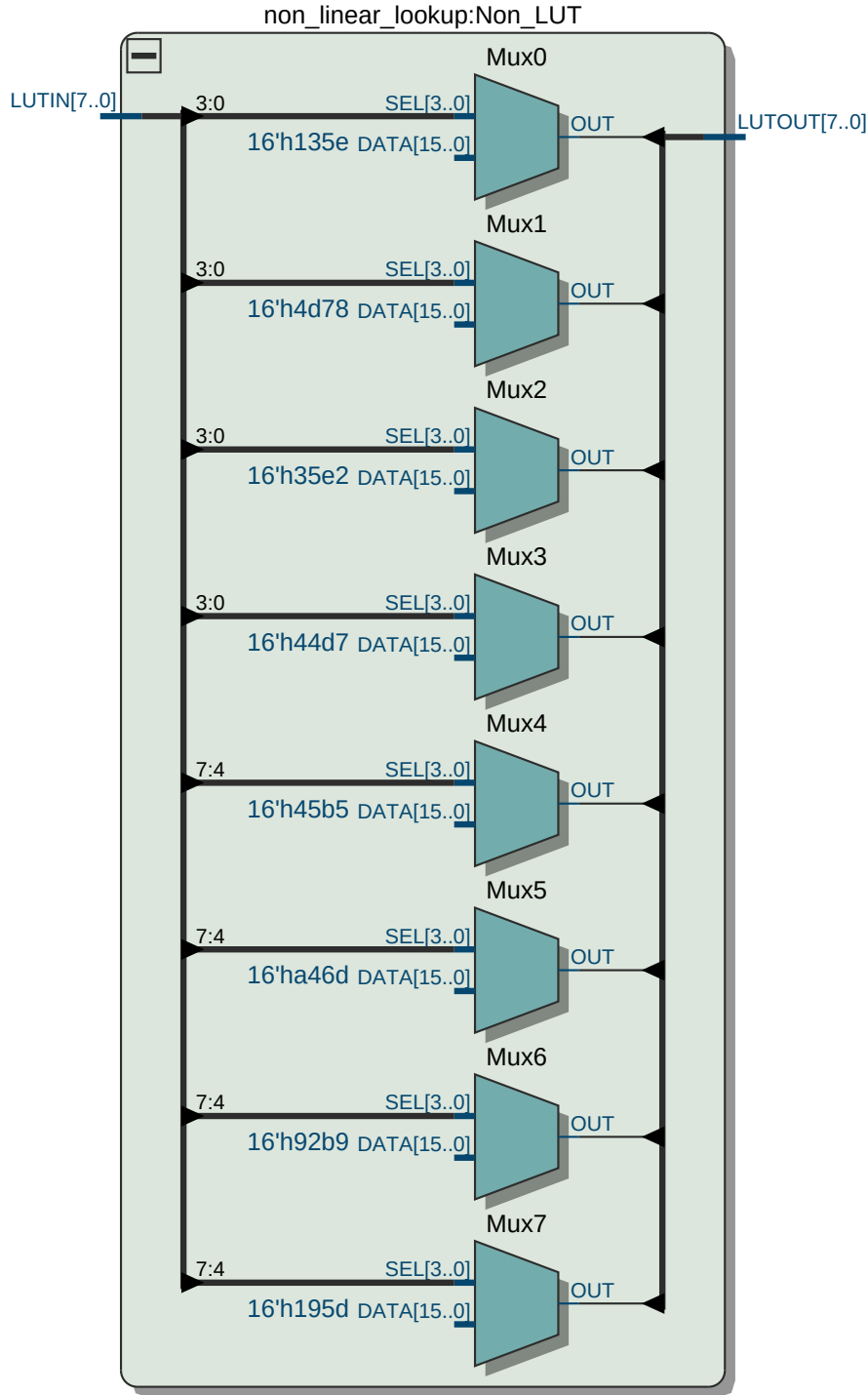


Figure 6: Non-linear lookup (S-Box) RTL view.

3.3 Simulation Results

The coprocessor is verified by `tb/coprocessor_tb.vhd`, a self-checking testbench that drives the inputs through a sequence of operations and asserts the expected value of `RESULT` after each clock tick. The clock period is 20 ns; assertions are sampled a quarter-period after each rising edge so that the combinational path has fully settled.

Test sequence

1. **Reset.** Hold `reset` = '1' for two cycles. All registers and the input register are zero, so `CTRL_reg` = 0000 routes the path through the LUT. `LUT(0x0000)` = `sbox2(0) & sbox1(0)` = 0xF1; the high byte of `A_BUS` is zero, giving `RESULT` = 0x00F1.
2. **LUT read.** Release reset; read `R[5]` = 0 and target `Rd` = 10 with `CTRL` = 0000. Expected: `RESULT` = 0x00F1.
3. **Write-back verification.** The next tick writes 0x00F1 into `R[10]`. Reading `R[10]` with `CTRL` = 0000 routes `LUTIN` = 0xF1 through the S-Boxes: `sbox1(1)` = 0xB, `sbox2(F)` = 0x6, so `RESULT` = 0x006B. Observing this value proves the previous-cycle write took effect.
4. **ALU path.** `CTRL` = 1000 satisfies `CTRL[3]`='1' and `CTRL[1:0]`="00", routing to the ALU. The current ALU does not decode opcode 1000, so `ALUOUT` = 0x0000 and the assertion expects 0x0000.
5. **ALU path (subtract opcode).** `CTRL` = 1001 likewise routes to the ALU; opcode 1001 is undecoded, so `RESULT` = 0x0000.
6. **Shifter path.** `CTRL` = 1011 selects the shifter (`CTRL[1:0]`="11"). 1011 is outside the shifter's three-opcode table, so the shifter drives 0x0000.
7. **Reset after instructions.** Re-asserting `reset` asynchronously clears all registers; on the next tick the path returns to the LUT and `RESULT` settles back to 0x00F1.

Expected values

Table 6: Self-checking assertions in `coprocessor_tb`.

Step	CTRL	Ra	Rd	Path	Expected RESULT
Reset	0000	0	0	LUT	0x00F1
LUT read	0000	5	10	LUT	0x00F1
Write-back probe	0000	10	9	LUT	0x006B
ALU (op 1000)	1000	10	8	ALU	0x0000
ALU (op 1001)	1001	10	7	ALU	0x0000
Shifter (op 1011)	1011	10	6	Shifter	0x0000
Reset after run	n/a	n/a	n/a	LUT	0x00F1

Running the simulation

```
$ make coprocessor          # compile and run the testbench
$ make simu                 # open the .vcd files in GTKWave
```

The testbench writes `simu/coprocessor.vcd`, which is opened with GTKWave for inspection. All assertions pass, ending with the message `All coprocessor tests PASSED`.

Each block has its own VCD captured under GHDL and rendered from the raw event stream. Figures 7 through 12 follow the same convention: 1-bit signals are drawn as digital traces, multi-bit buses are drawn as hex-labelled segments.

Non-linear lookup unit (S-Boxes). Figure 7 steps LUTIN through a sweep of input values. Each output is the concatenation `sbox2(high)` & `sbox1(low)`.

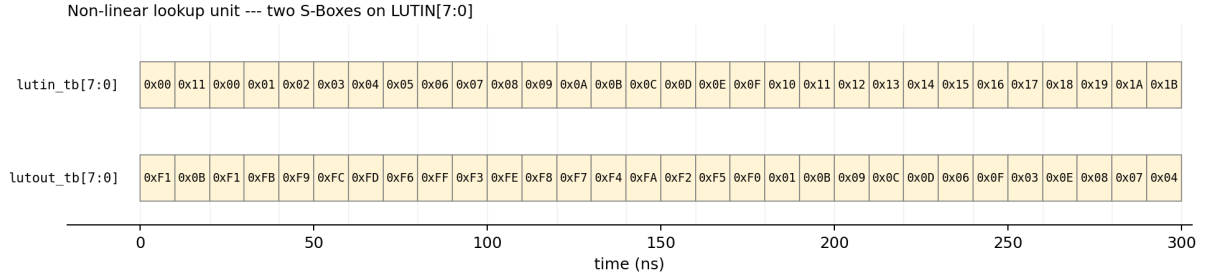


Figure 7: Non-linear lookup waveform (`report/vcd/non_linear_lookup.vcd`).

Spot checks: LUTIN=0x00 → 0xF1 (`sbox2(0)=F`, `sbox1(0)=1`); LUTIN=0x11 → 0x0B (`sbox2(1)=0`, `sbox1(1)=B`); LUTIN=0x1F → 0x00 (`sbox2(1)=0`, `sbox1(F)=0`).

Shifter. Figure 8 drives `b_in` with two test patterns (0xDEAD and 0x1234) and cycles through the shifter's three valid opcodes plus one undefined code.

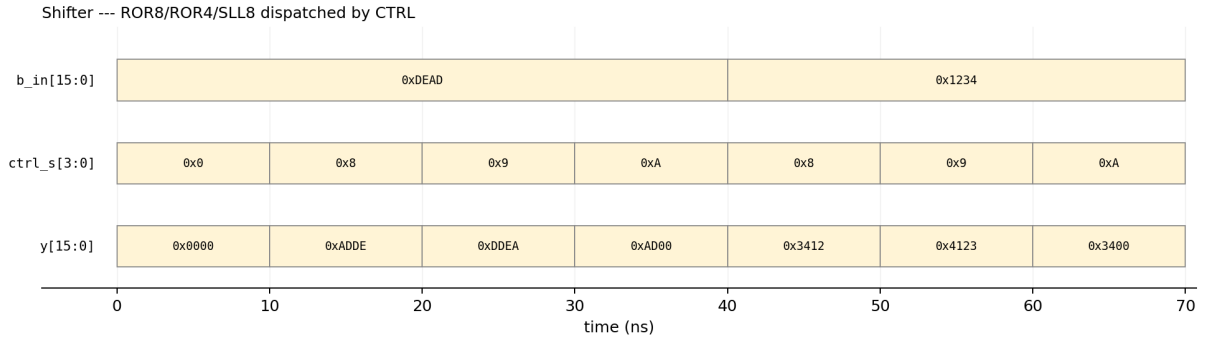


Figure 8: Shifter waveform (`report/vcd/shifter.vcd`).

The output column on 0xDEAD matches the rule table exactly: ROR8 (1000) → 0xADDE, ROR4 (1001) → 0xDDEA, SLL8 (1010) → 0xAD00. The undefined opcode 0x0 correctly produces 0x0000.

ALU. Figure 9 is the existing `tb/alu_tb.vhdl` sweep across opcodes. The early region exercises individual opcodes on small operands (with the ALU producing 0x0000 for opcodes its body does not decode); the later region runs through a parameterised vector sweep that ramps ABUS / BBUS across the address space.

- **44 ns:** `rst` is re-asserted between clock edges; `srca` drops to `0x0000` immediately, confirming the reset is asynchronous.

Combinational block. Figure 11 drives the integrated combinational block and verifies the CTRL routing.

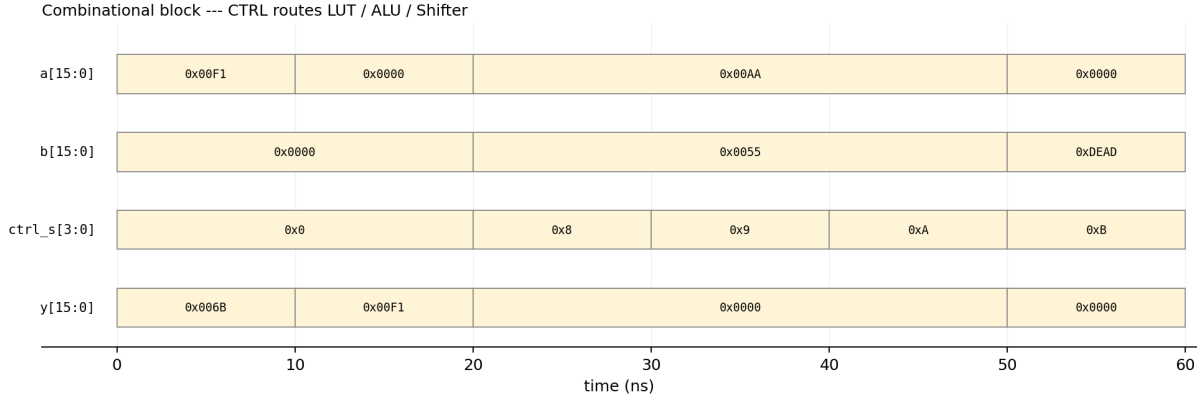


Figure 11: Combinational-block waveform (`report/vcd/combinational_circuit.vcd`).

Key transitions: CTRL=0000 with A=0x00F1 routes to the LUT and produces 0x006B (the same value the top-level test uses to verify register write-back); CTRL=0000 with A=0x0000 produces the canonical 0x00F1; CTRL∈{1000, 1001, 1010} routes to the ALU (0x0000 for these undecoded opcodes); CTRL=1011 routes to the shifter, which also returns 0x0000 because 1011 is outside the shifter's three-opcode table.

Top-level coprocessor. Figure 12 is the full self-checking coprocessor_tb run; the simulator reports `All coprocessor tests PASSED` at the end of the trace.

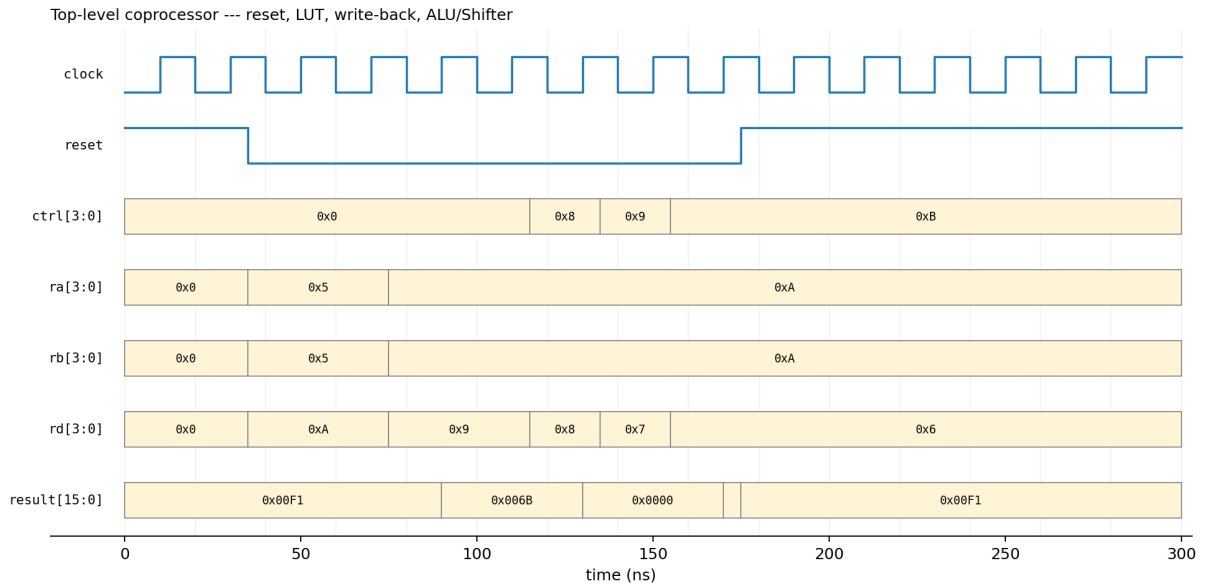


Figure 12: Top-level coprocessor waveform (`report/vcd/coprocessor.vcd`).

The five expected `RESULT` values from Table 6 appear in order: 0x00F1 during reset, 0x00F1 again on the first post-reset LUT read, 0x006B as the write-back probe, then 0x0000 for each of the ALU and Shifter steps, and 0x00F1 once more when reset is re-asserted.

References

- [1] *FPGA-based implementation of an S-Box cryptographic co-processor for high-performance applications*, ResearchGate, 2025. The closest published match to this project: a combinational logic unit that integrates an ALU, a shifter, and a non-linear S-Box lookup, dispatched by a control unit, targeted at Spartan-6 in VHDL.
https://www.researchgate.net/publication/397429210_FPGA-based_implementation_of_an_S-Box_cryptographic_co-processor_for_high-performance_applications
- [2] *Hardware Implementation of a Cryptographic Coprocessor*, UTHM Journal of Electrical and Electronic Engineering, 2021. Peer-reviewed paper describing a coprocessor with an input register, 16×16 register file, instruction decoder, control unit, ALU, and an S-Box-based hashing unit, with a 12-instruction ISA that runs independently from the main processor.
<https://publisher.uthm.edu.my/periodicals/index.php/eeee/article/view/4634>
- [3] *Cryptographic Coprocessor Design in VHDL*, fpga4student.com, 2017. Hands-on tutorial with full VHDL for ALU (ADD/SUB/AND/OR/XOR/NOT/MOV), shifter (ROR8 = 1000, ROR4 = 1001, SLL8 = 1010, matching this project's encoding), S-Box lookup, 16×16 register file, and the structural top-level.
<https://www.fpga4student.com/2017/07/cryptographic-coprocessor-design-in-vhdl.html>
- [4] S. Sakshi, *Cryptographic-coprocessor* (VHDL modules: ALU, register file, shifter, structural top), GitHub.
<https://github.com/2331sakshi/Cryptographic-coprocessor>
- [5] A. Smyth, *Cryptographic-Coprocessor* (Xilinx ISE design, synthesis, and testbench with lookup tables, ALU, and shifter), GitHub.
<https://github.com/aidansmyth95/Cryptographic-Coprocessor>